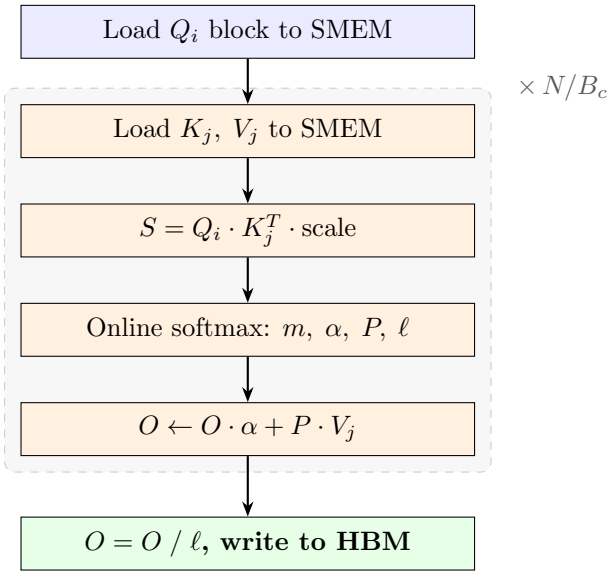
Flash Attention

From First Principles to GPU Code

Caleb Yenusah

Flash attention is an IO-aware reformulation of the attention computation. Instead of materializing the $N \times N$ score matrix in HBM, it tiles the computation so that softmax is applied incrementally and the output accumulates in registers. This document derives the algorithm from first principles and maps every equation to a self-contained CUDA kernel.



Contents

Notation

Symbol	Meaning	Typical Value
N	sequence length	1024–16384
d	head dimension	128
B_r	Q block rows (tile height)	64
B_c	KV block columns (tile width)	32
S	score matrix QK^T	$\mathbb{R}^{N\times N}$
P	attention weight matrix	$\mathbb{R}^{N\times N}$
O	output matrix	$\mathbb{R}^{N\times d}$
m_i	running row max (per row i)	scalar
ℓ_i	running row sum (per row i)	scalar
α	correction factor $e^{m_{\text{old}}-m_{\text{new}}}$	$\in (0, 1]$

Standard Attention

Given $Q, K, V \in \mathbb{R}^{N\times d}$ where N = sequence length and d = head dimension:

$$S = QK^T \in \mathbb{R}^{N\times N} \tag{1}$$

$$P = \text{softmax}\left(\frac{S}{\sqrt{d}}\right) \in \mathbb{R}^{N\times N} \tag{2}$$

$$O = PV \in \mathbb{R}^{N\times d} \tag{3}$$

Softmax is applied **row-wise**. For row i:

$$m_i = \max_j\left(\frac{S_{ij}}{\sqrt{d}}\right), \qquad \ell_i = \sum_j \exp\left(\frac{S_{ij}}{\sqrt{d}} - m_i\right), \qquad P_{ij} = \frac{\exp\left(\frac{S_{ij}}{\sqrt{d}} - m_i\right)}{\ell_i} \tag{4}$$

The subtraction by m_i is for numerical stability. It does not change the result:

$$\frac{\exp(x_j - c)}{\sum_k \exp(x_k - c)} = \frac{\exp(x_j)}{\sum_k \exp(x_k)} \quad \text{for any constant } c \tag{5}$$

Choosing $c = m_i = \max_j(x_j)$ ensures all exponents are ≤ 0, preventing overflow.

Why Naive Attention is Memory-Bound

A naive implementation launches three separate kernels, materializing S in HBM between each step:

All values in **elements** (multiply by 2 for FP16 bytes).

Step	Operation	HBM Reads	HBM Writes
1	$S = QK^T$	$2Nd$	N^2
2	$P = \text{softmax}(S/\sqrt{d})$	N^2	N^2
3	$O = PV$	$N^2 + Nd$	Nd
Total		$3N^2 + 3Nd$	$2N^2 + Nd$

Concrete Numbers ($N = 4096,\ d = 128$)

$$\text{FLOPs} = 4N^2 d = 4 \times 4096^2 \times 128 = 8.59 \text{ GFLOP}$$

$$\text{HBM traffic} \approx 5N^2 \times 2 \text{ bytes} = 5 \times 4096^2 \times 2 \approx 167 \text{ MB}$$

$$\text{Arithmetic Intensity}_{\text{naive}} = \frac{8.59 \text{ GFLOP}}{0.167 \text{ GB}} \approx 51 \frac{\text{FLOP}}{\text{byte}} \tag{6}$$

A100 ridge point (tensor core): 312 TFLOPS / 2039 GB/s $\approx$ 153 FLOP/byte. At 51 FLOP/byte, naive attention is **memory-bound** — tensor cores are starved waiting for HBM traffic caused by the $N \times N$ intermediate.

Flash Attention: IO Complexity

Flash attention **never materializes** the $N \times N$ score matrix in HBM. Instead, S is computed tile-by-tile in SMEM/registers, softmax is applied on-the-fly, and the result accumulates directly into O.

Operation	Elements
Read Q	Nd
Read K	Nd
Read V	Nd
Write O	Nd
Total	$4Nd$

$$\text{HBM traffic} = 4Nd \times 2 = 4 \times 4096 \times 128 \times 2 = 4.19 \text{ MB}$$

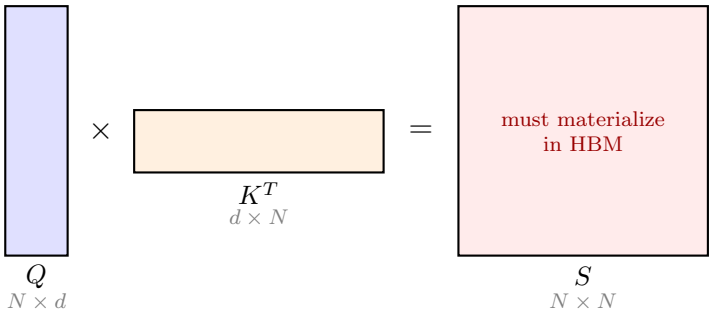
$$\boxed{\text{Arithmetic Intensity}_{\text{flash}} = \frac{8.59 \text{ GFLOP}}{0.00419 \text{ GB}} \approx 2050 \frac{\text{FLOP}}{\text{byte}}} \tag{7}$$

This is **13**$\times$ above the ridge point. Flash attention is firmly **compute-bound**. A **40**$\times$ reduction in HBM traffic versus naive.

The $N \times N$ intermediate S now lives entirely in SMEM/registers as small $B_r \times B_c$ tiles. The price: we need a way to compute softmax **incrementally** without seeing the full row of S at once.

Why the $N \times N$ Matrix is the Problem

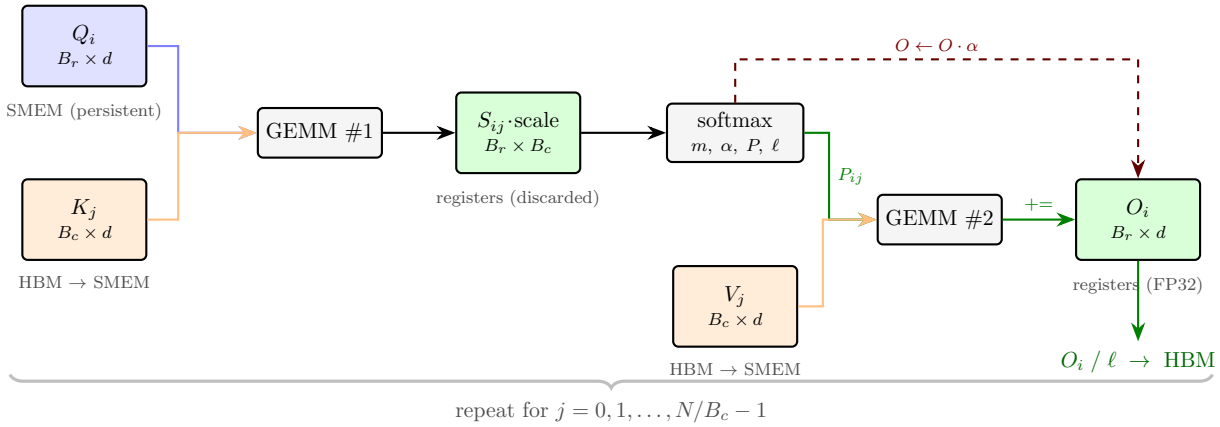
Standard attention computes $S = QK^T$ as a full $N \times N$ matrix before softmax can begin:



Flash attention avoids this by computing S one small tile at a time, applying softmax incrementally, and discarding each tile before computing the next.

Tiling Dataflow

Inside one thread block, processing one Q-block across all KV blocks:



Q_i stays in SMEM for the entire loop. Each iteration streams one K_j, V_j pair from HBM, computes the $B_r \times B_c$ score tile in registers (never written to HBM), applies online softmax with the α correction to O, and accumulates $P_{ij}V_j$ via GEMM #2. After all KV blocks, O_i is divided by ℓ and written to HBM once.

Online Softmax

Standard softmax is a **two-pass** algorithm:

$$\text{Pass 1: } m = \max(x_1, \ldots, x_N) \qquad \text{Pass 2: } \ell = \sum_j \exp(x_j - m), \quad y_j = \frac{\exp(x_j - m)}{\ell}$$

This requires the **entire row**. In flash attention, we only have B_c elements at a time. We need to compute m and ℓ incrementally.

Derivation

Suppose we have processed block A (first B_c elements) and have:

$$m_A = \max_{j \in A}(x_j), \qquad \ell_A = \sum_{j \in A} \exp(x_j - m_A)$$

Now block B (next B_c elements) arrives. The new global max is:

$$m_{\text{new}} = \max(m_A,\ \max_{j \in B}(x_j))$$

The old terms were computed with base m_A. We need to **re-base** them to m_{new}:

$$\exp(x_j - m_A) \cdot \underbrace{\exp(m_A - m_{\text{new}})}_{\alpha} = \exp(x_j - m_{\text{new}})$$

Define the **correction factor**:

$$\boxed{\alpha = \exp(m_{\text{old}} - m_{\text{new}})} \tag{8}$$

Since $m_{\text{new}} \geq m_{\text{old}}$, we have $\alpha \leq 1$ — always scaling **down**, numerically stable.
The updated running sum:

$$\boxed{\ell_{\text{new}} = \ell_{\text{old}} \cdot \alpha + \sum_{j \in B} \exp(x_j - m_{\text{new}})} \tag{9}$$

Proof:

$$\ell_{\text{new}} = \sum_{j \in A \cup B} \exp(x_j - m_{\text{new}}) = \underbrace{\sum_{j \in A} \exp(x_j - m_A)}_{\ell_A} \cdot \alpha + \sum_{j \in B} \exp(x_j - m_{\text{new}}) = \ell_A \cdot \alpha + \sum_{j \in B} \exp(x_j - m_{\text{new}}) \quad \checkmark$$

Output Accumulator Rescaling

We maintain a running **unnormalized** output accumulator:

$$O_{\text{unnorm}} = \sum_{j \text{ seen}} \exp(x_j - m_{\text{current}}) \cdot v_j$$

When the max changes, every previous term needs re-basing by the same factor α:

$$\boxed{O_{\text{new}} = O_{\text{old}} \cdot \alpha + \underbrace{\sum_{j \in B} \exp(x_j - m_{\text{new}}) \cdot v_j}_{P_{\text{block}} \times V_{\text{block}}}} \tag{10}$$

The second term is a **matrix multiply**: the exponentiated scores P_{block} times the value matrix V_{block}.

After all blocks: $O_{\text{final}} = O_{\text{unnorm}} \,/\, \ell$.

Worked Example

Let $\mathbf{x} = [1,\ 0,\ 2,\ 0]$ split into blocks $A = [1,\ 0]$ and $B = [2,\ 0]$.

Standard (needs entire row):

$m = 2, \quad \ell = e^{-1} + e^{-2} + e^{0} + e^{-2} = 0.368 + 0.135 + 1.0 + 0.135 = 1.638$

Online (streaming):

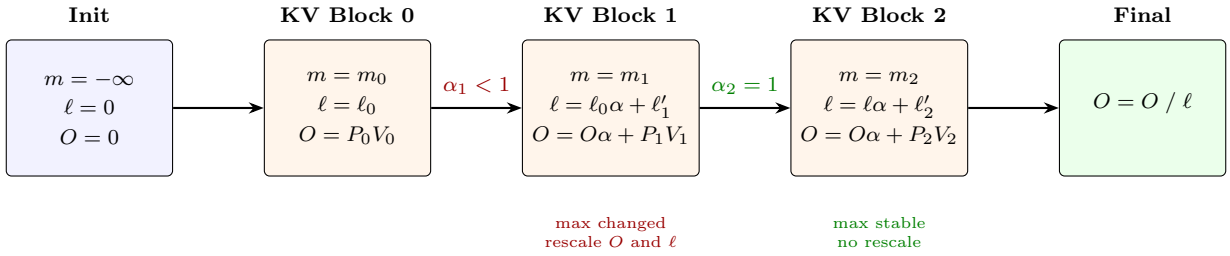
Init: $m = -\infty, \quad \ell = 0$

Block A: $m_{\text{new}} = \max(-\infty, 1, 0) = 1$
$\alpha = e^{-\infty - 1} = 0$ (first block, no history to correct)
$\ell = 0 \cdot 0 + e^{1-1} + e^{0-1} = 1.0 + 0.368 = 1.368$
$m = 1$

Block B: $m_{\text{new}} = \max(1, 2, 0) = 2$
$\alpha = e^{1-2} = 0.368$ (max changed — rescale)
$\ell = 1.368 \cdot 0.368 + e^{2-2} + e^{0-2} = 0.503 + 1.0 + 0.135 = \mathbf{1.638}$ ✓
$O = O_{\text{old}} \cdot 0.368 + P_B \cdot V_B$
$m = 2$

Both produce $\ell = 1.638$. The online version corrected its previous work by $\alpha = 0.368$ when the max changed.

State Evolution Across KV Blocks



- α is a per-row **scalar** applied to a **matrix** O — cheap rescale of expensive accumulator.
- Once the running max stabilizes (common after a few blocks), $\alpha = 1$ and no rescale is needed.
- The denominator ℓ is only used at the very end — we carry unnormalized O throughout.

The Complete Update Rule

At each KV block j, for each query row i:

$$
\begin{aligned}
S_j &= Q_i\, K_j^T && \text{(GEMM \#1)} \\
m_{\text{new}} &= \max(m,\ \text{rowmax}(S_j/\sqrt{d})) && \text{(per-row max)} \\
\alpha &= \exp(m - m_{\text{new}}) && \text{(correction, } \leq 1\text{)} \\
P_j &= \exp(S_j/\sqrt{d} - m_{\text{new}}) && \text{(element-wise)} \\
\ell &\leftarrow \ell \cdot \alpha + \text{rowsum}(P_j) && \text{(update denominator)} \\
O &\leftarrow O \cdot \alpha + P_j\, V_j && \text{(GEMM \#2 + rescale)} \\
m &\leftarrow m_{\text{new}}
\end{aligned}
\tag{11}
$$

Initialization: $m = -\infty,\ \ell = 0,\ O = 0$. Finalization: $O = O\ /\ \ell$.

Algorithmic Verification (Python)

```python
for kv_start in range(0, N, Bc):
    K_block = K[kv_start : kv_start + Bc]    # [Bc, d]
    V_block = V[kv_start : kv_start + Bc]    # [Bc, d]

    S = Q_block @ K_block.T * scale           # GEMM #1: [Br, Bc]

    m_new = np.maximum(m, S.max(axis=1))      # per-row max
    alpha = np.exp(m - m_new)                 # correction factor

    P = np.exp(S - m_new[:, None])            # exponentiate

    l = l * alpha + P.sum(axis=1)             # update denominator
    O = O * alpha[:, None] + P @ V_block      # GEMM #2 + rescale

    m = m_new

O_final = O / l[:, None]                      # normalize
```

Causal Masking

In autoregressive models, token i can only attend to tokens $j \leq i$. This is enforced by a **lower-triangular mask**:

$$
\tilde{S}_{ij} = \begin{cases} S_{ij} & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}
$$

Since $\exp(-\infty) = 0$, masked positions contribute nothing to ℓ or O. The online softmax algorithm requires **zero changes**.

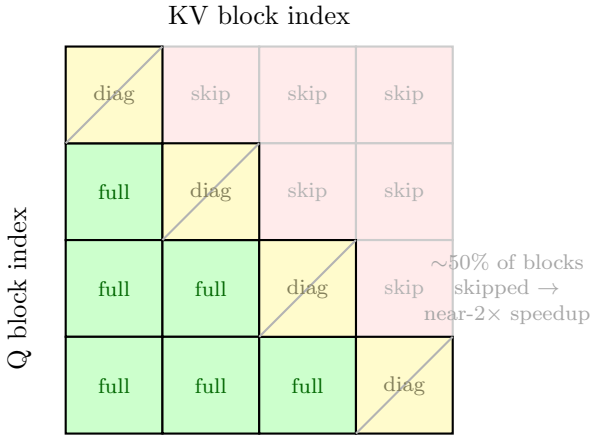
Block-Level Classification

The Q block covers rows $[q,\ q + B_r)$ and the KV block covers columns $[k,\ k + B_c)$. Three cases:

$$q + B_r \leq k \implies \text{entire tile is masked} \rightarrow \textbf{skip} \tag{12}$$

$$q \geq k + B_c \implies \text{entire tile is visible} \rightarrow \textbf{full} \tag{13}$$

$$\text{otherwise} \implies \text{diagonal tile} \rightarrow \textbf{element-wise mask} \tag{14}$$



Skipping upper-triangle blocks avoids $\sim N^2/2$ elements of unnecessary compute. The element-wise mask on a diagonal block sets $S[\text{local\_q}][\text{local\_k}] = -\infty$ wherever global_q $<$ global_k.

CUDA Core Implementation

This section maps the algorithm to a self-contained CUDA kernel using pure scalar FP32 arithmetic. No tensor cores, no MMA, no `ldmatrix`, no `cp.async` — just `FMUL` + `FADD` on CUDA cores. The kernel is a readable reference, not a performance target.

Source: `src/cuda_core_reference.cu`.

Thread and Block Mapping

One thread per output row. `NUM_THREADS` $= B_r$. Each thread independently computes its row's QK dots, softmax, and PV accumulation. Threads cooperate only on SMEM loads.

```
template<int BR, int BC, int D>
__global__ void flash_cuda_core(
    const __half* __restrict__ Q,
    const __half* __restrict__ K,
    const __half* __restrict__ V,
    __half* __restrict__ O,
    int N)
{
    const float SCALE = 1.0f / sqrtf((float)D);

    int tid   = threadIdx.x;        // row index within Q-block
    int q_blk = blockIdx.x;        // which Q-block
    int head  = blockIdx.y;
```

```
int batch = blockIdx.z;

int base = (batch * gridDim.y + head) * N * D;
```

Grid: `dim3(N/BR, n_heads, batch)`. Each block processes one $B_r \times d$ output tile.

SMEM Layout

All shared memory is FP32. FP16 $\rightarrow$ FP32 conversion happens at the GMEM/SMEM boundary.

$$\text{SMEM} = \underbrace{B_r \times d}_{Q} + \underbrace{B_c \times d}_{K} + \underbrace{B_c \times d}_{V} = (B_r + 2B_c) \times d \times 4 \text{ bytes} \tag{15}$$

For $B_r = 64, B_c = 32, d = 128$: $(64 + 64) \times 128 \times 4 = 64\,\text{KB}$.

```
extern __shared__ float smem[];
float* sQ = smem;                        // [BR x D]
float* sK = smem + BR * D;               // [BC x D]
float* sV = smem + (BR + BC) * D;        // [BC x D]

// Load Q tile once: GMEM (FP16) -> SMEM (FP32)
for (int i = tid; i < BR * D; i += BR)
    sQ[i] = __half2float(Q_tile[i]);
__syncthreads();
```

Q is loaded once and reused for every KV block. K and V are overwritten each iteration.

Per-Thread Accumulators

Direct mapping to the math: `O_acc[D]` $\leftrightarrow O$, `m` $\leftrightarrow m$, `l` $\leftrightarrow \ell$.

```
float O_acc[D];
for (int d = 0; d < D; d++)
    O_acc[d] = 0.0f;              // O = 0
float m = -INFINITY;               // m = -inf
float l = 0.0f;                    // l = 0
```

Main Loop

Each phase maps directly to the update rule from Section 6.

```
int n_kv = N / BC;

for (int j = 0; j < n_kv; j++) {

    // Cooperative load K[BC x D] and V[BC x D]
    const __half* Kj = K_head + j * BC * D;
    const __half* Vj = V_head + j * BC * D;
    for (int i = tid; i < BC * D; i += BR) {
        sK[i] = __half2float(Kj[i]);
        sV[i] = __half2float(Vj[i]);
    }
    __syncthreads();
```

Phase 1 — $S_j = Q_i K_j^T / \sqrt{d}, \quad m_{\text{new}} = \max(m, \text{rowmax}(S_j))$

```
        float S[BC];
        float new_max = m;
        for (int c = 0; c < BC; c++) {
            float dot = 0.0f;
            for (int d = 0; d < D; d++)
                dot += sQ[tid * D + d] * sK[c * D + d];
            S[c] = dot * SCALE;
            new_max = fmaxf(new_max, S[c]);
        }
```

Each thread computes its row's dot product with all B_c key vectors. The inner loop over `D` is the contraction dimension of GEMM #1. Row max is found in the same pass.

Phase 2 — $\alpha = \exp(m_{\text{old}} - m_{\text{new}}), \quad O \leftarrow O \cdot \alpha, \quad \ell \leftarrow \ell \cdot \alpha$

```
        float alpha = __expf(m - new_max);
        for (int d = 0; d < D; d++)
            O_acc[d] *= alpha;
        l *= alpha;
        m = new_max;
```

When m does not change, $\alpha = 1$ and this is a no-op.

Phase 3 — $P_j = \exp(S_j - m), \quad \ell \mathrel{+}= \text{rowsum}(P_j), \quad O \mathrel{+}= P_j V_j$

```
        for (int c = 0; c < BC; c++) {
            float p = __expf(S[c] - m);
            l += p;
            for (int d = 0; d < D; d++)
                O_acc[d] += p * sV[c * D + d];
        }

        __syncthreads();
    }
```

The inner loop over `D` is GEMM #2, unrolled: for each column c of P, we scale $V_j[c, :]$ by $P[c]$ and accumulate into O. The `__syncthreads()` at the bottom protects SMEM before the next KV block overwrites it.

Epilogue

```
    float inv_l = 1.0f / l;
    for (int d = 0; d < D; d++)
        O_tile[tid * D + d] = __float2half(O_acc[d] * inv_l);
}
```

Normalize O by ℓ, convert FP32 $\rightarrow$ FP16, write to GMEM. Each thread writes its own row.

Item	Type	Cost
`O_acc[D]`	d floats in registers	128 regs
`S[BC]`	B_c floats in registers	32 regs
`m, l, alpha, p, dot, ...`	scalars	∼10 regs
Total per thread		∼170 / 255 max
`sQ[BR × D]`	SMEM (FP32)	32 KB
`sK[BC × D]`	SMEM (FP32)	16 KB
`sV[BC × D]`	SMEM (FP32)	16 KB
Total SMEM		**64 KB**

Table 1: *
$B_r = 64,\ B_c = 32,\ d = 128.$

Resource Budget

Summary

Concept	Key Equation / Code
Standard attention	$O = \text{softmax}(QK^T/\sqrt{d}) \cdot V$
Naive AI	~ 51 FLOP/byte (memory-bound)
Flash AI	~ 2050 FLOP/byte (compute-bound)
Correction factor	$\alpha = \exp(m_{\text{old}} - m_{\text{new}})$ → `__expf(m - new_max)`
Running sum	$\ell \leftarrow \ell \cdot \alpha + \text{rowsum}(P)$ → `l = l*alpha + ...`
Output rescale	$O \leftarrow O \cdot \alpha + PV$ → `O_acc[d] *= alpha; O_acc[d] += p*sV[...]`
Causal block skip	skip if $q_{\text{off}} + B_r \leq k_{\text{off}}$
SMEM footprint	$(B_r + 2B_c) \times d \times 4$ bytes
Register budget	$\sim d + B_c + 10$ floats per thread